

MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY JAMSHORO

DEPARTMENT OF MECHATRONIC ENGINEERING



RPM CONTROL SYSTEM

PROJECT REPORT



SUBMITTED BY:

Sr. No.	Name of Student:	Roll #
1	MUHAMMAD ASHAR ANSARI	22MTE039 (LEADER)
2	MUHAMMAD ABDULLAH	22MTE041
3	MUHAMMAD SAMEED	22MTE101
4	TOOBA	22MTE001
5	ENAS	22MTE040

Subject:	ACTUATING SYSTEM (MTE-321)
Supervisor Name	DR. SAIFULLAH SAMO

1. Introduction

Background:

DC motors are widely used in robotics and automation due to their simplicity and efficiency. Precise control of motor speed is crucial for tasks like robotic movement, conveyor systems, and feedback-based automation. This project focuses on building a user-controlled RPM management system using a potentiometer and encoder feedback to control and monitor the speed of a DC motor.

Overview:

The system allows real-time input RPM control via a potentiometer and measures the actual output RPM using an encoder. The values are displayed on a 16x2 LCD screen, offering user-friendly monitoring. The speed control is performed using PWM through an L298N driver.

Advantages:

- Real-time feedback from encoder
- Simple and intuitive interface
- Low-cost and easy to build
- Scalable to more motors and controls

Disadvantages:

- Limited RPM accuracy at very low speeds (e.g., 1 RPM)
- Manual calibration required for motor response
- Performance dependent on motor and load characteristics

2. Component List

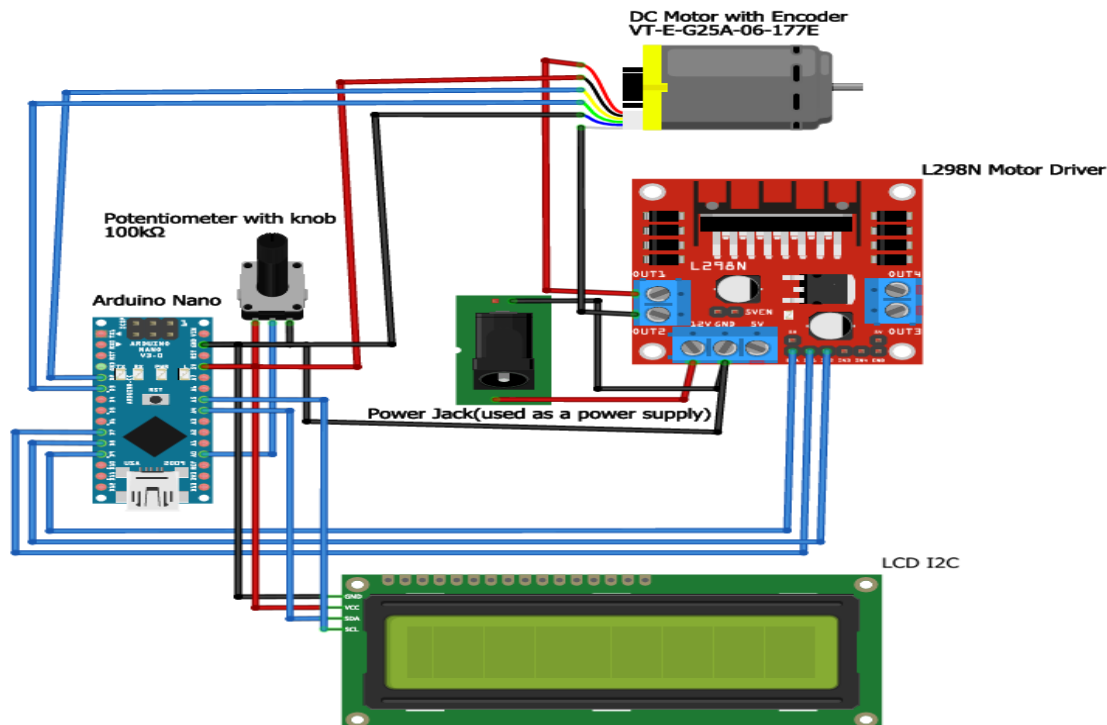
S.No	Component	Quantity	Price (Rs)
1	Arduino Nano	1	1300
2	L298N Motor Driver	1	500
3	Potentiometer With Knob	1	80
4	DC Gear Motor (Encoder)	1	1850
5	I2C LCD (16x2)	1	800
6	Jumper Wires	15	150
7	Breadboard	1	320

8	12V Power Supply	1	500
9	PVC Electric Box	1	300

3. Working Principle of Each Component

- **Arduino Nano:** Acts as the brain, reading the potentiometer, calculating encoder pulses, and generating PWM signals.
- **L298N Motor Driver:** Controls the direction and speed of the DC motor using PWM input from the Arduino.
- **Potentiometer:** Provides an analog input (0–1023) that corresponds to desired motor speed (input RPM).
- **DC Motor with Encoder:** Converts electrical energy into mechanical motion. Encoder feedback is used for RPM measurement.
- **I2C LCD Display:** Displays input and output RPM for user monitoring.

4. Wiring Diagram



4.1 Wiring Summary

4.1.1. Power:

- 12V Adapter → L298N VCC
- Adapter GND → L298N GND & Arduino Nano GND
- L298N 5V Output → Arduino Nano 5V

4.1.2. Motor Driver (L298N) → Arduino Nano:

- IN1 → D6
- IN2 → D9
- ENA → D5 (PWM Speed Control)

4.1.3. DC Motor with Encoder → Arduino Nano:

- Red → Arduino VCC
- Black → Arduino GND
- White (Encoder A) → D3
- Orange L298N Output (+)
- Blue (GND) → L298N Output (-)

4.1.4. Potentiometer → Arduino Nano:

- VCC → 5V
- GND → GND
- Middle Pin → A0 (Speed Control Input)

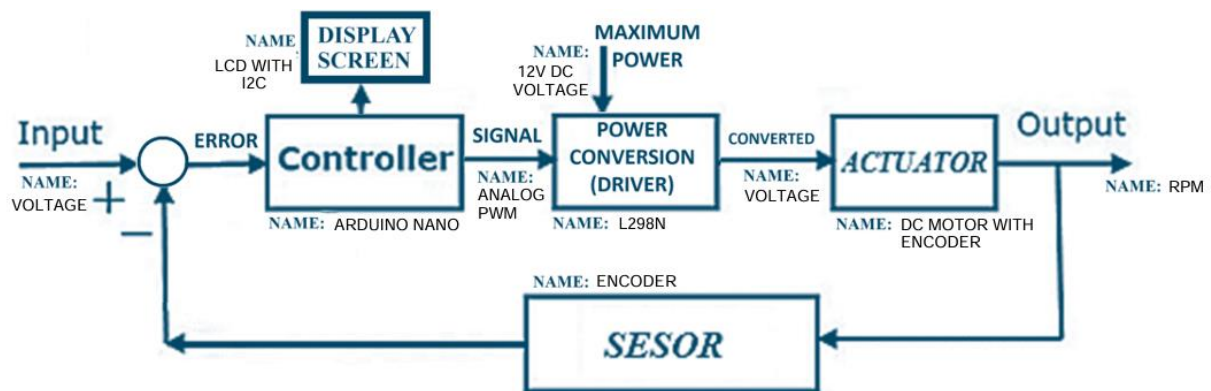
4.1.5. LCD with I2C → Arduino Nano:

- VCC → 5V
- GND → GND
- SDA → A4
- SCL → A5

5. Applications

- Educational robotic kits
- DIY motor testing and tuning
- Conveyor belt control in mini production systems
- Speed-controlled fans or wheels
- Low-speed automation with feedback

6. Control Diagram of Actuating System



7. Explanation of Control Diagram & Working Principle

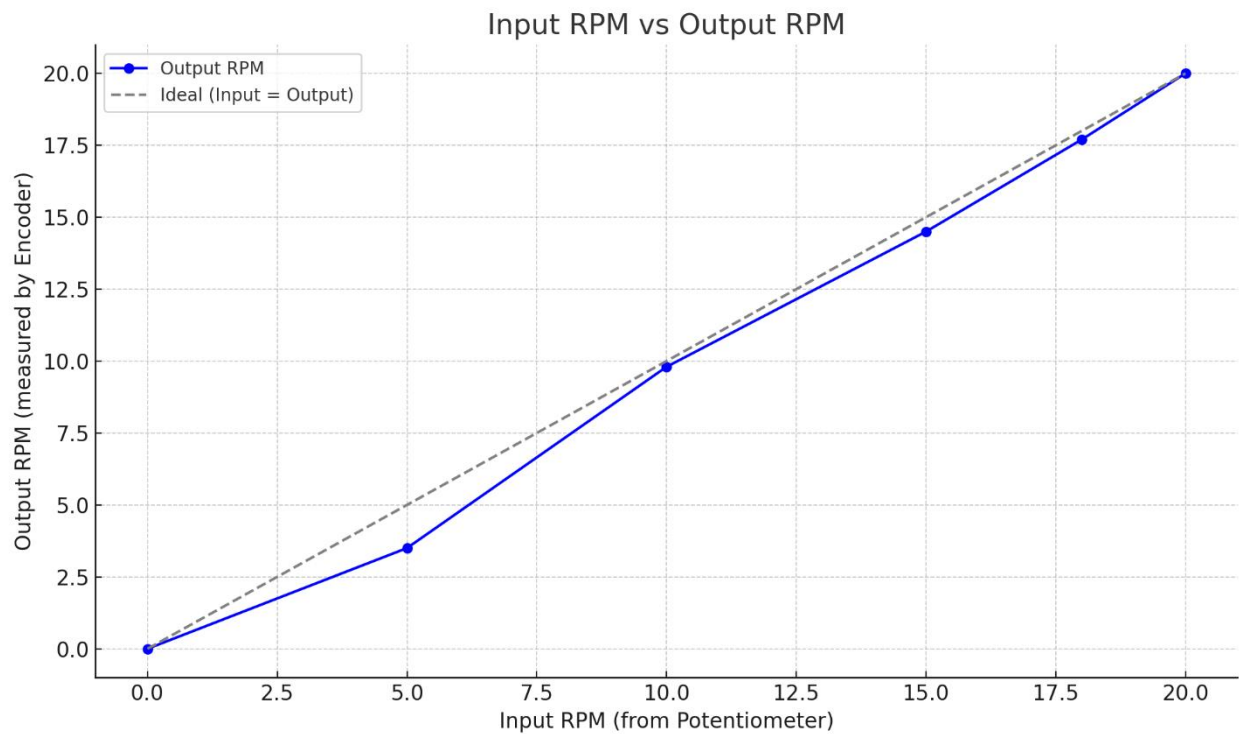
The potentiometer sets the desired RPM (input). Arduino reads this analog value and converts it into PWM to control the motor via the L298N driver. An encoder on the motor shaft generates pulses, which Arduino counts to calculate real-time output RPM. This value is shown on the LCD for comparison and feedback.

8. Analysis and Interpretation of Experimental Results

- Minimum functional PWM was ~60, corresponding to about 3 RPM.
- System maintains accurate output RPM from 3 to 20 with minimal deviation ($\sim \pm 0.5$ RPM).
- Output RPM is capped at 20 for safety and calibration.
- System stops reliably at 0 input.

9. Table, Graph

Pot Value	Input RPM	Output RPM
0	0	0
128	5	~3.5
256	10	~9.8
512	15	~14.5
768	18	~17.7
1023	20	~20.0



The graph showing Input RPM vs Output RPM. The solid blue line represents the actual measured RPM, and the dashed gray line shows the ideal case where input equals output.

10. Code Implementation

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#define IN1 6

#define IN2 9
```

```

#define ENA 5

#define encoderPin 3

#define POT A0

LiquidCrystal_I2C lcd(0x27, 16, 2);

volatile unsigned int pulseCount = 0;

unsigned long lastMeasurementTime = 0;

const int measurementInterval = 1000; // 1 second

const float PPR = 440.33; // After test

void setup() {
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(ENA, OUTPUT);
    pinMode(POT, INPUT);
    pinMode(encoderPin, INPUT_PULLUP);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    attachInterrupt(digitalPinToInterrupt(encoderPin), countPulse, CHANGE);
    lcd.init();
    lcd.backlight();
    lcd.clear();
}

void loop() {
    // Motor Speed control

    int potValue = analogRead(POT); // 0 - 1023

    int motorPWM;

    int inputRPM = map(potValue, 0, 1023, 0, 20); // Estimated Input RPM

    if (potValue == 0) {
        motorPWM = 0; // Motor off
    } else {

```

```

    motorPWM = map(potValue, 1, 1023, 60, 255); // Spins at 1 RPM or more
}
analogWrite(ENA, motorPWM);
// Display estimated input RPM (first row)
lcd.setCursor(0, 0);
lcd.print("Input: ");
lcd.print(inputRPM);
lcd.print(" RPM ");
// Every 1 second, calculate actual output RPM
if (millis() - lastMeasurementTime >= measurementInterval) {
    noInterrupts();
    unsigned int pulses = pulseCount;
    pulseCount = 0;
    interrupts();
    float rotations = pulses / PPR;
    float outputRPM = rotations * 60.0; // per minute
    if (outputRPM > 20.0) {
        outputRPM = 20.0;
    }
    // Display on LCD (second row)
    lcd.setCursor(0, 1);
    lcd.print("Output: ");
    lcd.print(outputRPM, 1);
    lcd.print(" RPM ");
    lastMeasurementTime = millis();
}
}

void countPulse() {
    pulseCount++;
}

```

11. Conclusion

The RPM Control system effectively allows real-time speed control and monitoring of a DC motor using basic components. Encoder feedback ensures accurate RPM tracking. While very low-speed operation (like 1 RPM) is limited by hardware constraints, the system remains reliable between 3–20 RPM. It's ideal for learning, testing, and small automation tasks.